Table des matières

CSIV — Banque de questions prédictives (révision examen) **1**
Générée avec la méthodologie BrainMax — archétypes Explain / Predict / Refactor / Debug 1
Domaine détecté : Codage d'image JPEG (Exercices 1–5) 1
Domaine détecté : Codage vidéo MPEG (Exercices 6–8) 2
Domaine détecté : Théorie de l'information (Cours 1–2) 3
Domaine détecté : Flux optique (Cours 3) 3
Comment utiliser cette banque 4

CSIV — Banque de questions prédictives (révision examen)

Générée avec la méthodologie BrainMax — archétypes Explain / Predict / Refactor / Debug

Chaque question est **ancrée dans le code réel du TP** (`Compression_Techniques/`) ou dans les supports de cours. Barème par question : 0 (aucune compréhension) ⟶ 1 (reconnaissance) ⟶ 2 (application) ⟶ 3 (maîtrise, peut enseigner).

Domaine détecté : Codage d'image JPEG (Exercices 1–5)

Question 1 — Explain (Exercice 1)

Dans `CodageEntropique.m`, la ligne `f = f(f > 0);` filtre les fréquences nulles avant le calcul de `H = -sum(f .* log2(f))`. **Pourquoi cette ligne est-elle indispensable ? Que se passerait-il sans elle ?**

> **Corrigé (2–3 attendus) :** `accumarray` ne produit des zéros que si certains symboles n'apparaissent pas… en fait ici tous les indices viennent de `unique`, donc chaque classe existe — mais le filtrage reste une sécurité générique. Sans lui, $\log_2(0) = -\infty$ et $0 \times (-\infty) = \text{NaN}$: l'entropie deviendrait `NaN` dès qu'une classe a une fréquence nulle. C'est une protection contre le cas dégénéré (vecteur vide, ou généralisation future). **Point clé :** $0 \cdot \log_2(0)$ doit être traité comme 0 dans la définition de l'entropie (limite).

Question 2 — Explain (Exercice 3)

Dans `ChoixTableQuantification.m`, la table luminance de base $T^{(l)}(50)$ est asymétrique : $T_{1,2} = 11$ mais $T_{2,1} = 12$. **Pourquoi les valeurs ne sont-elles pas symétriques, alors que la DCT d'un bloc est une base symétrique ?**

> **Corrigé (2–3) :** La table n'est pas mathématiquement dérivée de la DCT — elle est **calibrée sur le système visuel humain** (études psycho-visuelles, norme JPEG 1992). L'asymétrie 11 vs 12 reflète une sensibilité légèrement différente entre fréquences horizontales et verticales. La table encode « ce que l'œil tolère de perdre », pas une propriété de la transformée.

Question 3 — Predict (Exercice 3)

`QuantificationDCT.m` fait `bloc_transforme = round(bloc ./ T_Quant)` en sens Direct et `round(bloc .* T_Quant)` en sens Inverse. **Si on supprime le `round` du sens Inverse uniquement, le PSNR final change-t-il ? L'image reconstruite est-elle identique ?**

> **Corrigé (2–3) :** Le `round` inverse n'a **aucun effet** sur les valeurs quantifiées (elles sont déjà des entiers × entiers = entiers… sauf si `T_Quant` contient des non-entiers après adaptation FQ). Après l'adaptation $T^{(FQ)} = \text{round}\left(\frac{50 \cdot T^{(50)}}{FQ}\right)$, la table est arrondie une fois pour toutes dans `ChoixTableQuantification` — donc les produits sont déjà entiers et le `round` inverse est un no-op. PSNR inchangé. **Piège classique :** le round qui fait perdre l'information est celui du Direct, pas celui de l'Inverse.

Question 4 — Refactor (Exercice 4)

`CoefficientsACDC` construit `Coeff_AC = [Coeff_AC coeff_ac(1:non_nul)]` dans une boucle sur tous les blocs. **Quel est le problème de performance de cette écriture et comment la corriger sans changer la sortie ?**

> **Corrigé (2–3) :** La concaténation croissante est $O(n^2)$: chaque `[Coeff_AC ...]` réalloue et recopie tout le vecteur. Pour une image 512 × 512 (4096 blocs, ≈ 12 500 coefficients), c'est négligeable ; pour de la vidéo UHD c'est un tueur

de performances. **Correction** : préallouer (estimation haute) puis tronquer, ou collecter dans un tableau de cellules + `horzcat` final, ou deux passes (compter puis remplir).

Question 5 — Debug (Exercice 4 — VRAI bug potentiel)

Dans `CompressionJPEG.m`, le poids est calculé par `CodageEntropique(Coeff_AC)` où `Coeff_AC` contient le marqueur **EOB = 1000**. **Quel inconvénient théorique ce choix introduce-il sur l'estimation du poids ? Est-ce un bug ou un choix acceptable ?**

Corrigé (2–3) : Le 1000 est un symbole **artificiel très fréquent** (un par bloc) : son code entropique est très court, donc son impact sur le poids est faible. MAIS il viole l'hypothèse « symboles quelconques » : si une image avait naturellement des coefficients quantifiés égaux à 1000 (possible avec FQ très élevé et résidus élevés), le EOB serait **indistinguable d'un vrai coefficient** → décodage impossible. En JPEG réel, on utilise un codage (run, size) Huffman dédié, pas une valeur magique. **Choix acceptable pédagogiquement, incorrect en production.**

Domaine détecté : Codage vidéo MPEG (Exercices 6–8)

Question 6 — Explain (Exercice 6)

`EstimationMouvement.m` initialise `MVr = zeros(ceil(n/k), ceil(m/k), 2)` avec `k = 16`. **Pourquoi `ceil` et pas `floor` ? Que contiennent les derniers blocs si l'image n'est pas multiple de 16 ?**

Corrigé (2–3) : `ceil` garantit que **chaque pixel appartient à un macro-bloc** même si 16 ne divise pas n ou m. Les derniers blocs sont alors **partiels** : `lignes = i:min(i+k-1, n)` les tronque. Conséquence dans `PredictionImage` : le vecteur de mouvement s'applique à un bloc plus petit — la prédiction reste correcte car les indices de référence sont clampés aux mêmes bornes.

Question 7 — Explain (Exercice 6)

`EQMMacrocBlocVoisin` renvoie `EQM = inf` si le macro-bloc décalé sort de l'image de référence. **Pourquoi `inf` plutôt que 0 ? Quel est l'effet sur l'algorithme CSA ?**

Corrigé (2–3) : `inf` rend la position **hors-image systématiquement perdante** au `min(EQM)` : le CSA ne peut jamais s'y déplacer. Avec 0, l'algorithme pourrait « téléporter » le bloc hors de l'image dès la première itération (EQM nul battant tout). C'est la façon propre d'implémenter une contrainte de bord sans `if` de branchement.

Question 8 — Predict (Exercice 6)

Dans `CSAMacroBloc`, la boucle s'arrête quand `mouvement == 1`, c'est-à-dire quand le minimum des 5 EQM est **le centre**. **Si deux voisins ont exactement le même EQM minimal (ex. haut et gauche), que se passe-t-il ? Le résultat est-il optimal ?**

Corrigé (2–3) : `[~, mouvement] = min(EQM)` renvoie le **premier** indice du minimum → le centre (indice 1) gagne les égalités, puis haut (2), gauche (3), droite (4), bas (5). L'algorithme s'arrête dès que le centre est (co)minimal → il peut s'arrêter prématurément sur un **minimum local** alors qu'un autre chemin descendait plus bas. C'est intrinsèque au CSA : c'est une descente gloutonne, pas une recherche exhaustive. Le résultat n'est **pas garanti optimal** (cf. bonus 8 voisins).

Question 9 — Refactor (Exercice 7)

`CompressionMPEG.m` compresse `Ir_Originale` en JPEG **à chaque appel**, or dans un vrai GOP, l'image de référence est codée une seule fois pour plusieurs images courantes. **Proposez une restructuration de l'API pour éviter ce re-codage.**

Corrigé (2–3) : Séparer le cycle de vie : `Ir_Codee = CompressionJPEG(Ir, ...)` une fois par GOP, puis passer `Ir_Codee` (ou `Ir_Decodee`) à `CompressionMPEG` au lieu de `Ir_Originale`. Attention au piège : la prédiction doit alors se faire sur **l'image décodée** (avec ses artefacts JPEG), pas l'originale — sinon le résidu estimé côté encodeur diffère de celui reconstruit côté décodeur (dérift de référence, « drift error »). C'est exactement ce que fait `DecompressionMPEG` : il prédit depuis `Ir_Decodee`.

Question 10 — Debug (Exercice 8 — VRAI subtilité)

`DecompressionMPEG.m` fait `Ic_Decodee = Ires_Decodee + Ip` où `Ip` est prédite depuis `Ir_Decodee` (référence **décompressée**). Mais `CompressionMPEG.m` **calcule le résidu avec `Ip` prédite depuis `Ir_Originale` (référence** non compressée**). Pourquoi cette asymétrie ne fait-elle pas exploser le PSNR ?**

> **Corrigé (2–3) :** Elle **dégrade légèrement** le PSNR (erreur de drift), mais ne l'explose pas car : (1) la décompression JPEG de `Ir` est fidèle (PSNR élevé à FQ raisonnable), donc $I_d^{Ref} \approx I^{Ref}$; (2) l'erreur de prédiction résiduelle est absorbée par… le résidu lui-même, qui est codé séparément. Les encodeurs MPEG réels **corrigent** ceci en faisant la prédiction sur la référence reconstruite localement (« local decode ») pour que l'erreur soit identique des deux côtés. Le TP s'en sort car la table Résidu (16 uniforme) tolère cette erreur.

Domaine détecté : Théorie de l'information (Cours 1–2)

Question 11 — Explain

Le cours affirme que l'entropie `H(S)` est « le plus petit nombre de bits en moyenne pour coder un symbole que l'on pourra déchiffrer à coup sûr ». **Pourquoi « en moyenne » ? Pourquoi pas « pour chaque symbole » ?**

> **Corrigé (2–3) :** Un code sans perte peut donner des codes de longueurs **différentes** par symbole (Huffman) : les symboles fréquents courts, les rares longs. La limite H est atteinte **en moyenne** sur un long message — un symbole individuel peut coûter plus que H bits. « À coup sûr » = décodage unique (code préfixe). Un code fixe à $\lceil \log_2 K \rceil$ bits par symbole est sans perte mais $> H$ dès que la source n'est pas uniforme.

Question 12 — Predict

Une image a une entropie de 7,2 bits/pixel mais est stockée en uint8 (8 bits/pixel). **Quel est le taux de compression maximal sans perte ? Peut-on atteindre 50% ?**

> **Corrigé (2–3) :** Max sans perte $= 1 - \frac{7{,}2}{8} = \mathbf{10}\,\%$. Non, 50 % est impossible sans perte : la limite de Shannon est $H = 7{,}2$ bits/pixel, soit un poids minimum de $\frac{7{,}2}{8} = 90\,\%$ de l'original. Pour dépasser, il faut accepter des **pertes** (quantification) — c'est exactement le passage $H_0 \to H_P$ du diagramme débit-distorsion.

Question 13 — Predict

On applique le filtre PNG **Sub** (différence avec le pixel de gauche) sur une image contenant un **gradient vertical** parfait (chaque colonne constante, variation verticale de 1 par ligne). **Quel filtre (None/Sub/Up/Average/Paeth) minimise l'entropie des différences ici ? Pourquoi ?**

> **Corrigé (2–3) :** Le filtre **Up** : la différence avec le pixel du haut vaut exactement 1 partout ⟶ suite constante ⟶ entropie quasi nulle après codage. Sub donnerait la valeur du pixel elle-même (pas de corrélation horizontale) ⟶ aucune réduction. Average et Paeth s'en sortiraient aussi (prédiction ≈ correcte) mais Up est optimal car la variation verticale est parfaitement prévisible. C'est le principe : **le filtre doit correspondre à l'orientation de la corrélation**.

Domaine détecté : Flux optique (Cours 3)

Question 14 — Explain

Le cours présente Horn & Schunck (1981) :

$$\min_{v} \; \frac{1}{2} \left\| I(x + v_x,\, y + v_y,\, t + 1) - I(x, y, t) \right\|_2^2 \; + \; \frac{\lambda}{2} \left(\|\nabla v_x\|_2^2 + \|\nabla v_y\|_2^2 \right)$$

Quel est le rôle du terme $\frac{\lambda}{2} \left(\|\nabla v_x\|_2^2 + \|\nabla v_y\|_2^2 \right)$? Que se passe-t-il si $\lambda \to 0$?

> **Corrigé (2–3) :** C'est un **a priori de régularité** (champ de mouvement lisse). Sans lui ($\lambda \to 0$), le problème est mal posé — **problème de l'ouverture** : une équation, deux inconnues (v_x, v_y) par pixel ; on ne perçoit que le mouvement perpendiculaire aux contours. $\lambda \to 0$ donne un champ de mouvement bruité/incohérent, sensible au bruit d'image. λ grand sur-lisse et perd les discontinuités (objets qui se chevauchent).

Question 15 — Predict

L'hypothèse de linéarisation

$$I(x + v_x,\ y + v_y,\ t + 1) \ \approx \ I(x, y, t + 1) + v_x \frac{\partial I}{\partial x} + v_y \frac{\partial I}{\partial y}$$

suppose des **déplacements petits**. **Que se passe-t-il si un objet se déplace de 20 pixels entre deux images ? La méthode différentielle échoue-t-elle et pourquoi le block-matching (TP) y résiste ?**

> **Corrigé (2–3) :** La linéarisation (développement de Taylor ordre 1) est invalide pour de grands déplacements : le gradient local ne décrit pas une variation de 20 px. Les méthodes différentielles divergent ou trouvent $v \approx 0$. Le block-matching (CSA du TP) est une **recherche discrète** : il teste des positions explicites (±1 px par itération, jusqu'au minimum local) et n'utilise aucune hypothèse de continuité — il converge vers le bon déplacement tant qu'il ne tombe pas dans un minimum local intermédiaire. C'est le compromis différentiel (dense, sous-px) vs patch (par bloc, entier).

Comment utiliser cette banque

1. **Auto-test :** répondre à voix haute ou par écrit, puis comparer au corrigé.
2. **Barème :** 0 = à côté ; 1 = bon concept sans mécanisme ; 2 = mécanisme correct, arêtes manquantes ; 3 = mécanisme + limites + alternatives.
3. **Règle :** une réponse correcte sans justification = 1. Le raisonnement vaut plus que le résultat.
4. Questions 5, 10 : vraies subtilités de conception — si vous les réussissez, vous êtes au niveau « enseigner ».
